

System Detective

CIE-1, Computer Organisation & Operating Systems (CO1) | TE IT | Roll No. _____ | Name Hardik Bhalekar
Submitted September 2026

1. The machine under inspection

Every number in this report came from my own Linux environment running through WSL2 on Windows. I am stating the virtualisation layer up front because it affects how the processor and memory are presented to Linux. The environment reports a four-logical-CPU Intel Core i3-4010U at 1.70 GHz, with two physical cores and two threads per core. It also reports the Microsoft hypervisor. The machine has 6.1 GiB of memory and 2.0 GiB of configured swap. The five required system tools were used on the same environment, and the locality experiment was measured separately. The dashboard was then built around the same live data rather than typed values.

Figure 1. Lscpu output showing the x86_64 architecture, four logical CPUs, two cores, two threads per core, Microsoft virtualization and cache hierarchy. Insert the captured Lscpu screenshot here.

2. Instruction set and what it tells me

The machine reports an x86_64 ISA. It is little-endian, meaning the least significant byte of a multi-byte value is stored at the lowest address. The CPU exposes a 48-bit virtual address size and a 39-bit physical address size. The difference matters because programs operate through virtual addresses while the operating system and memory management hardware translate those addresses to physical memory.

The flags include SSE, AVX and AVX2 extensions. AVX2 is a SIMD extension, so suitable instructions can operate on multiple data values in parallel. The cache hierarchy reported by Lscpu is 64 KiB of L1 data cache across two instances, 64 KiB of L1 instruction cache, 512 KiB of L2 and 3 MiB of L3. These cache levels are much smaller than main memory, so the locality experiment later in the report is a direct demonstration of why access order matters.

3. Memory at the moment of capture

The free -h output showed 6.1 GiB total memory, 313 MiB used, 5.0 GiB free, 1.1 GiB in buff/cache and 5.8 GiB available. Swap was 2.0 GiB, with 0 B currently used.

The important distinction is between free and available. Free is memory that is currently unused, while available also includes memory that the kernel can reclaim when an application needs it. The 1.1 GiB buff/cache therefore is not simply lost memory. Linux uses available RAM for caching because cached data can often be discarded and recreated when necessary.

Figure 2. free -h output. The captured values are 6.1 GiB total, 313 MiB used, 5.0 GiB free, 1.1 GiB buff/cache, 5.8 GiB available and 2.0 GiB swap. Insert the captured free -h screenshot here.

4. Processes, threads and states

ps -eLf displays one row for each lightweight process/thread entry. In my capture, PID 1 appears with two LWPs, showing that the /init process has two kernel-scheduled threads. The shell and the ps/head commands also appear as separate entries. The important distinction is that threads belonging to one process share the process address space and resources, while each thread can be scheduled independently.

The htop view was used to observe CPU, memory and process activity interactively. Because the environment exposes four logical CPUs, the CPU view can show activity across four logical processors. Process state information from ps is also reflected in the dashboard. In general, S represents interruptible sleep, R represents running or runnable work, and I identifies an idle kernel thread.

Figure 3. htop/ps process information captured from the WSL environment. Insert the captured screenshot here.

5. System calls and the user/kernel boundary

strace -c was used to observe the system calls made by a command. This is useful because a normal program does not perform every operation entirely in user space. Operations such as opening files, reading directory entries, querying terminal information and creating or mapping memory require interaction with the kernel.

The important idea from the trace is not just the number of calls, but the boundary being crossed. A user-space instruction can execute without entering the kernel, whereas a system call transfers control to kernel code so that the operating system can perform a protected operation on behalf of the process.

Figure 4. strace -c output for the selected command, showing the system-call summary. Insert the captured strace -c screenshot here.

6. Locality of reference: the experiment

I used an 8192 x 8192 two-dimensional array and compared row-major and column-major traversal. The arithmetic performed by both loops is the same; only the order in which memory is accessed changes. In row-major traversal, the inner loop follows the storage order of a C array, so consecutive iterations access nearby addresses. In column-major traversal, the next element is separated by an entire row.

Run	Row-major (s)	Column-major (s)	Ratio
1	0.101057	0.551465	5.46×
2	0.091092	0.503329	5.52×
3	0.091402	0.524849	5.74×
Mean	0.094517	0.526548	5.57×

The mean row-major time was 0.094517 s while the mean column-major time was 0.526548 s, giving a measured slowdown of about 5.57× for column-major traversal. The reason is spatial locality. A cache line contains multiple nearby bytes, so sequential row access can use several values after one line is fetched. Column access repeatedly jumps by a large stride, reducing the useful data obtained from each cache line. Both loops perform the same order of arithmetic work, so the difference comes from the memory access pattern rather than algorithmic complexity.

The result also shows why a simple cache-line calculation should not be treated as an exact timing prediction. Real execution includes hardware prefetching, cache hierarchy effects, translation overhead and the behaviour of the processor under the virtualised environment. The measured value therefore has to be established experimentally.

7. The dashboard

The dashboard was implemented as a Streamlit application. It combines the required system information into one view instead of requiring the user to inspect several terminal commands separately. The CPU section reads processor information from `lscpu`, the memory section uses `psutil`, the process section lists the top five processes with their states, and the locality section displays the measured timing results.

The important design choice is that the dashboard is based on live system information. CPU and memory values are obtained from the current Linux environment and the process table can change between refreshes. The locality experiment is measured separately because running the complete benchmark on every dashboard refresh would add unnecessary delay. The dashboard therefore acts as a single observation layer over the command-line data and the benchmark result.

Figure 5. Final dashboard showing CPU/ISA information, memory usage, top processes and locality timing. Insert the final dashboard screenshot here.

8. What the measurements show

The command outputs and experiment connect several Computer Organisation and Operating Systems concepts to one working system. The `lscpu` output identifies the ISA, logical processors, cache hierarchy and virtualisation layer. `free -h` shows how the operating system divides physical memory between active use, unused memory and reclaimable cache. `ps -eLf` demonstrates the relationship between processes and threads, while `strace` makes the user/kernel boundary visible. Finally, the locality experiment shows that two loops with the same number of operations can have different execution times because they interact differently with the memory hierarchy.

9. Honesty and reproduction

The numerical values in this report were taken from the captured Linux command output or from the locality benchmark. The memory and process values are momentary observations and can change when the system is observed again. The locality timing can also vary with processor load and the virtualised environment.

To reproduce the work, run the required commands in the WSL2 Linux environment, compile and execute the locality program with the GCC compiler, save its three timing runs, and then start the Streamlit dashboard from the project environment. The current environment uses GCC 15.2.0 and `strace` 6.19. The dashboard dependencies used during development include Streamlit, `psutil` and `pandas`.

The accompanying AI interaction log should be read together with this report. AI was used for conceptual explanations and development assistance, but machine-specific values were verified against actual command output and measurements rather than assumed from a typical computer.